INVENTORY MANAGEMENT

MR.PIBHU CHITBURANACHART
MR.PANNAWISH KRIENGYAKUL
MR.AKKHRAWIN NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

INVENTORY MANAGEMENT

MR.PIBHU CHITBURANACHART
MR.PANNAWISH KRIENGYAKUL
MR.AKKHRAWIN NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

Computer Engineering Project
entitled

INVENTORY MANAGEMENT

Mr.Pibhu Chitburanachart
Researcher

Mr.Pannawish Kriengyakul
Researcher

Mr.Akkhrawin Nair
Researcher

Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science),
Ph.D. (Computer Engineering)
Advisor

Thesis
entitled
INVENTORY MANAGEMENT

was submitted to the Faculty of Engineering & International College,
Mahidiol University
for the degree of Bachelor of Engineering (Computer Engineering)
on
July 15, 2024

_____
Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science),
Ph.D. (Computer Engineering)
Chair Committee

_____
Committee 1, Ph.D. (Computer Engineering)
Committee

_____
Committee 2, Ph.D. (Computer Engineering)
Comittee

BIOGRAPHY

NAME	Mr.Pibhu Chitburanachart
DATE OF BIRTH	29 August 2004
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	098 494 7984
E-MAIL	Pibhu22@gmail.com

NAME	Mr.Pannawish Kriengyakul
DATE OF BIRTH	17 April 2003
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	0989705604
E-MAIL	peto03.tcs@gmail.com

BIOGRAPHY

NAME	Mr.Akkhrawin Nair
DATE OF BIRTH	22 September 2002
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	0987819644
E-MAIL	akkhrawin22@gmail.com

ACKNOWLEDGEMENT

First and foremost, we would like to express our special thanks and sincere of gratitude to our advisor and co-advisors, Asst. Prof. Mingmanas Sivaraksa, Asst. Prof. Lalita Narupiyakul, Asst. Prof. Tanasanee Phienthrakul, who insight, advise and knowledge us a lot in finalizing this project within the limited time frame.

Beside our advisors, we would like to thank to the committees, Asst. Prof. XXX XXXX, Asst. Prof. YYY YYYY, and Asst. Prof. ZZZ ZZZZ, for their insightful comment and encouragement. Also, the questions which encourage us to rethink and research more about some factors which we missed. Therefore, this project would not be completed without comments, questions, and support from our committees.

Finally, we would like to special thanks to our university, Mahidol University International College, and Faculty of Engineering, Mahidol University, which provided us a lot of resources to study, financial means for researching this project.

Mr.Pibhu Chitburanachart
Mr.Pannawish Kriengyakul
Mr.Akkhrawin Nair

ชื่อโครงงาน	ชื่อวิทยานิพนธ์
ผู้วิจัย	นายพิภู จิตบูรณะชาติ ICCI
	นาย ปัณณวิชญ์ เกรียงยะกุล ICCI
	นายอัครวินทร์ แนร์ ICCI
	ปริญญา วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
อาจารย์ที่ปรึกษา	ผศ.□□. มิ่งมานัส ศิวรักษ์
สำเร็จการศึกษา	15 กรกฎาคม 2567

บทคัดย่อ

บทคัดย่อภาษาไทย (not required for your project proposal, but it is mandatory for the black book)

คำสำคัญ : ลาเท็ก / วิทยานิพนธ์ (~5 คำ)

39 หน้า

INVENTORY MANAGEMENT

STUDENTS	Mr. Pibhu Chitburanachart ICCI
	Mr. Pannawish Kriengyakul ICCI
	Mr. Akkhrawin Nair ICCI
DEGREE	Bachelor of Engineering (Computer Engineering)
PROJECT ADVISOR	Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science), Ph.D. (Computer Engineering)
DATE OF GRADUATION	July 15, 2024

ABSTRACT

This project presents the design and implementation of an Inventory Management System for small and medium-sized trading businesses. The system is intended for middle-man business workflows in which a company purchases products from suppliers, manages received stock, sells products to customers, and monitors operational documents related to receivables and payables.

The system was developed as a three-tier web application using React and Vite for the frontend, Django and Django REST Framework for the backend, and MySQL for relational data storage. The implementation includes master data management for products, categories, suppliers, and customers; purchase and sales transaction management; quotation records; billing notes; payment batches; credit notes; stock validation; dashboard summaries; and a read-only text assistant for selected operational questions.

The backend is responsible for authoritative business logic. Available stock is derived from received purchase line items minus committed sales line items rather than being manually stored as a static product value. Sales are validated server-side before they can move into stock-deducting statuses, and stock allocation is performed through received purchase layers using FIFO logic. Finance workflows are also controlled by backend eligibility endpoints so that billing notes, payment batches, and credit notes can only be created from valid transactions.

The result is a functional web-based inventory management system that integrates purchasing, sales, stock visibility, and financial document tracking in one application. The assistant feature is currently limited to stock and fulfillment questions, customer or supplier summaries, receivable and payable exception checks, and document reference or line-item lookup. The system supports daily SME trading operations and provides a foundation for future improvements such as expanded inventory planning formulas, role-based permissions, formal audit logs, and larger-scale production testing.

KEYWORDS : Inventory Management / SME Trading / Django REST Framework / React / MySQL / AI Assistant

39 Pages

CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

1.1 Background

Small and medium-sized trading businesses often operate as intermediaries between suppliers and customers. Their daily operations depend on purchasing goods from suppliers, receiving stock, selling goods to customers, preparing business documents, and monitoring the margin between purchase cost and sales revenue. In such businesses, inventory management is not only a matter of counting product quantities. It also requires the coordination of quotations, purchase orders, sales records, fulfillment status, receivables, payables, and stock availability.

Many small businesses still depend on spreadsheets, paper records, manual document folders, or general-purpose accounting tools. These methods may be workable for a small number of products, but they become difficult to control when the business must track multiple suppliers, product units, transaction statuses, cancelled lines, delayed purchases, customer billing, and supplier payments. Manual workflows also make it difficult to answer operational questions quickly, such as which products are low in stock, which sales can be billed, which purchases are ready for payment, or whether a sale can be fulfilled from available received stock.

The Inventory Management System was developed to address these practical workflow problems for an SME trading environment. The implemented system uses a React frontend, a Django REST Framework backend, and a MySQL relational database. Its design focuses on master data management, purchasing, sales, quotations, billing notes, payment batches, credit notes, dashboard summaries, stock validation, and read-only text queries for selected operational information. The backend is responsible for authoritative validation and stock calculations, while the frontend provides a compact web interface for daily operational work.

The project also includes an AI-assisted inventory chat feature with a limited scope. It is a read-only helper that can summarize supported backend data; it does not cre-

ate, edit, approve, cancel, or override transactions. Its current supported areas are stock and fulfillment questions, customer or supplier summaries, receivables and payables with exception checks, and reference or line-item lookup.

1.2 Objective

1. To develop a system that records purchase and sales transactions, including the current status of each process, with an attachment file function that links all supporting documents directly to the corresponding transactions.
2. To develop an inventory management system that displays the overall product stock in the warehouse and provides tools for calculating essential inventory information.
3. To integrate an AI chatbot using the ChatGPT API that connects to the system's backend data, allowing users to request supported operational summaries through text queries.

1.3 Scope

The scope of this project is limited to the implemented Inventory Management System codebase. The system covers the following functional areas:

1. **Master data management:** The system manages categories, products, suppliers, and customers. Products support SKUs, previous SKUs, product names, category links, unit conversions, reorder levels, active or disabled status, product pictures, and supplier-specific sourcing information.
2. **Purchasing workflow:** The system records purchase transactions, supplier information, purchase line items, expected delivery dates, received dates, item statuses, VAT totals, bill discounts, supplier tax invoice references, uploaded documents, and payable totals. Purchase status and purchase item status are synchronized by backend logic.
3. **Sales workflow:** The system records sales transactions, customer information, customer purchase-order references, line items, delivery statuses, uploaded documents, VAT totals, discounts, billing-note links, credit-note links, and source

quotation links. Stock is validated on the server before a sale can move into a stock-deducting status.

4. **Inventory and stock control:** Available stock is derived from received purchase quantities minus committed sale quantities. Sale allocations use received purchase layers and support first-in, first-out allocation logic. The inventory workspace and dashboard show stock position, reorder information, demand, purchase pipeline, stock value, and product history.
5. **Quotation workflow:** The system stores quotations with customer and supplier references, quotation validity information, shipping date, payment term, VAT mode, total amounts, normalized line items, and supplier options for quotation items. Quotations can be linked to derived purchases and sales.
6. **Finance workflow:** The system supports billing notes for customer receivables, payment batches for supplier payables, and credit notes for cancelled or returned sales lines. Eligibility endpoints prevent users from selecting transactions that should not be included in these finance documents.
7. **Dashboard and reporting:** The backend provides dashboard summary and segment endpoints for operational summaries such as finance, trends, product movement, cashflow, and order coverage.
8. **Authentication and access flow:** The backend includes JWT login, refresh, and authenticated user profile endpoints. The frontend supports login, token refresh, logout, and guest mode. Backend-wide authentication enforcement is controlled by the INVENTORY_REQUIRE_AUTH setting.
9. **AI-assisted operation:** The system includes a text-based inventory assistant endpoint. Its current scope is read-only support for stock and fulfillment questions, customer or supplier summaries, receivables and payables with exception checks, and reference or line-item lookup.

1.4 Expected Results

The expected result of this project is a functional web-based inventory management system that can record purchase and sales transactions, track the status of each process, and link supporting documents directly to their corresponding transactions. The

system is expected to provide an up-to-date view of stock levels based on received purchase quantities and committed sale quantities, together with clear traceability between products, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes.

The system is also expected to improve operational accessibility by providing dashboard summaries and a limited AI-assisted text interface. Users should be able to retrieve supported information such as product stock, selected transaction references, receivables, payables, partner summaries, and stock exceptions without manually searching every module. Overall, the completed system is expected to reduce manual checking, improve stock visibility, and support better day-to-day decision making for an SME trading business.

1.5 Timeline

Table 1.1 Project Timeline

Plan	Timeline								
	2024				2025				
	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May
1: Initiation									
2: Data Analyze									
3: Requirements									
4: Data Setup									
5: Development									
6: Integration									
7: User Test									
8: Final Draft									
9: Submittion									

CHAPTER 2
LITERATURE REVIEW

This chapter reviews three key works that support the development of this project. The first reference, A Full-Stack Web Solution for Online FMCG Management System Using MERN, gives useful background for building a web-based inventory platform with transaction records and document management. The second reference, Essentials of Inventory Management, supports the inventory workflow used in this project, especially the need to connect purchasing, sales, stock visibility, replenishment, and stock value. The third reference, Aisha: A Custom AI Library Chatbot Using the ChatGPT API, gives background for adding a chatbot-style interface that can answer questions from prepared system data. Together, these works support the technical, inventory, and assistant-related direction of the Inventory Management System.

2.1 A Full-Stack Web Solution for Online FMCG Management System Using MERN: Design, Implementation, and Evaluation [2]

This project presents the comprehensive development of a Full-Stack Web Solution for an Online Fast-Moving Consumer Goods (FMCG) Management System. Designed to modernize and streamline retail operations for FMCG products, the system leverages the power of the MERN stack (MongoDB, Express.js, React, and Node.js) for high availability, scalability, and data integrity. The article details the three core phases of the project: Design, Implementation, and Evaluation. The design phase focuses on developing an intuitive, responsive, and secure architecture, including multi-user role support (e.g., admin, staff, supplier) and key modules for inventory management, order processing, real-time stock updates, and automated billing. The implementation phase outlines the deployment of the MERN stack components, creating a robust backend for API integration and a dynamic frontend for a seamless user experience. Finally, the evaluation phase assesses the system's effectiveness in enhancing operational efficiency,

reducing manual effort, and bridging the gap between traditional retail methods and modern digital solutions, catering to the needs of retailers, wholesalers, and distributors.

2.2 Essentials of Inventory Management [3]

Timeless stock-keeping fundamentals meet up-to-the minute technologies to optimize efficiency and drive profits! Inventory management is about more than counting what you've got. It's about understanding business realities and making decisions that balance current demand with future needs—while keeping overhead and operating costs to a minimum. Now in its Second Edition, Essentials of Inventory Management gives inventory professionals the information they need to maximize productivity in key areas, from physical stock issues to problem identification and resolution to technologies like RFID and other automated inventory mechanisms. Perfect for novice and veteran managers alike, this ultra-practical book covers topics such as: Forecasting and replenishment strategies • Differences between retail and manufacturing inventories • Materials requirements planning and just-in-time inventory systems • Simple formulas for calculating quantities and schedules • Management of inventory as a physical reality and a monetary value • Supply chain risk management Complete with detailed examples, handy tools, and a revised and expanded chapter analyzing "Why Inventory Systems Fail and How to Fix Them," this nontechnical yet thorough guide is perfect for both instructional and on-the-job use.

2.3 Aisha: A Custom AI Library Chatbot Using the ChatGPT API [1]

This article describes the development of "Aisha," a custom chatbot for a university library, built using Python and the ChatGPT API. Aisha is designed to provide reference and support services (e.g., answering library queries) outside normal operating hours. The paper discusses the chatbot's architecture, capabilities, limitations, and the development process. It positions this as one of the early attempts to deploy a ChatGPT-based system for library services and provides insight into customization, prompt design, integration, and user feedback.

CHAPTER 3
METHODOLOGY

This chapter explains how the Inventory Management web application was developed. The focus is not only on the tools used, but also on how each page supports the real workflow of an SME trading business. In this project, we built the system as a React web application connected to a Django REST Framework backend and a MySQL database.

3.1 Development Approach

We developed the system step by step, starting from the data that every other workflow depends on. The first part was master data, such as products, categories, suppliers, and customers. After that, we developed the transaction modules: purchases, sales, and quotations. The finance-related modules, including billing notes, payment batches, and credit notes, were added after the main transactions were already working. Finally, we connected the dashboard, inventory workspace, and AI chat page to the backend data.

The project was separated into three main layers:

1. **Frontend layer:** React and Vite were used to build the web pages, forms, filters, tables, modals, and tab navigation.
2. **Backend layer:** Django REST Framework was used to expose APIs, validate data, calculate stock, control eligibility rules, and prepare dashboard and assistant responses.
3. **Database layer:** MySQL was used through the Django ORM to store relational business records.

This structure helped keep the user interface separate from the business rules. The frontend guides the user, but the backend is still responsible for the important checks such as stock validation, transaction eligibility, and payable amount synchronization.

3.2 Requirement Areas and Web-App Pages

The web application is organized around the main work areas that a trading business needs each day. Each sidebar page has a specific purpose, and the pages are connected through shared product, supplier, customer, purchase, sale, and finance data.

Table 3.1 Requirement Areas and Implemented Pages

Requirement Area	Web-App Page	Purpose in the System
Operation summary	Dashboard	Shows stock, finance, order coverage, trends, and movement summaries from backend data.
Inventory control	Inventory	Shows stock health, reorder information, supplier options, product details, and quick purchase-order preparation.
Assistant support	AI Chat	Answers supported read-only questions about stock, fulfillment, partners, AR/AP exceptions, and document references.
Purchasing	Purchases	Records purchase orders, purchase items, receiving status, documents, VAT totals, discounts, and payable values.
Supplier payment	Payment Batches	Groups eligible purchases into supplier payment batches and tracks paid or unpaid lines.
Quotation	Quotation	Prepares quotations with line items, supplier options, validity dates, and conversion to purchase or sale records.
Sales	Sales	Records customer sales, line items, stock allocation, delivery status, documents, VAT totals, and credit-note links.
Customer billing	Billing Notes	Groups eligible sales into customer billing notes and tracks received or outstanding lines.
Returns and cancellation adjustment	Credit Notes	Creates credit notes from eligible cancelled or returned sale items.
Product records	Products	Manages product details, SKUs, pictures, units, reorder levels, active status, suppliers, and history.
Category records	Categories	Manages product categories using a parent-child category tree.
Supplier records	Supplier	Manages supplier profile data, contact information, procurement contact, branches, shipping addresses, and terms.

3.3 Development of Each Page

3.3.1 Dashboard Page

The dashboard page was developed as the first screen for checking the business situation quickly. It receives backend dashboard data and shows several summary areas, including stock health, reorder planning, stock cycling, delivery pipeline, purchase planning, popular products, finance, cashflow, and order coverage. The dashboard is also connected to other pages. For example, a low-stock product can lead the user to the inventory or product page, and an order issue can lead to the purchase or sales page.

3.3.2 Inventory Page

The inventory page was developed as the main stock-control workspace. It uses the backend stock report rows instead of calculating stock from full transaction lists in the browser. The page includes an inventory control board, a searchable directory, filters for health status, category, supplier, value, and reorder need, product detail views, and a quick purchase-order drawer. This page is important because it gives the user a practical view of what is available, what is incoming, and what may need to be reordered.

3.3.3 AI Chat Page

The AI chat page was developed as a read-only helper. The user can type a question or use example prompts. The backend prepares a compact context from the current system data and returns a structured answer. If an OpenAI API key is configured, the backend can send the local summary and data context to the configured OpenAI model. If not, it returns the local summary. The current assistant scope is limited to four areas: stock and fulfillment, customer or supplier summaries, receivables/payables with exceptions, and reference or line-item lookup. It does not create, edit, approve, cancel, or delete any record.

3.3.4 Purchases Page

The purchases page was developed for supplier-side transactions. Users can create purchase records, add line items, attach documents, edit existing purchases,

update purchase status, and update item receiving status. Purchase items can be pending, received, or cancelled. Only received purchase items become available stock. The page also supports search, filters, pagination, VAT mode, item discounts, bill discount, expected delivery dates, received dates, and payment batch links.

3.3.5 Payment Batches Page

The payment batches page was developed for supplier payable tracking. Instead of letting users manually select any purchase, the page loads eligible purchases from the backend. A purchase is eligible only when it passes the backend rules, such as not already being attached to an active payment batch. Each payment batch stores supplier information, planned and actual payment dates, bank reference, total amount, and paid line status.

3.3.6 Quotation Page

The quotation page was developed for preparing offers before purchase or sales records are finalized. It supports quotation line items, supplier cost options, validity settings, shipping date, payment terms, VAT mode, and total values. The page can also start a purchase or sales flow from a quotation. In the database, quotation items are stored in the QuotationItem table, and supplier options are stored in QuotationItemSupplier; however, the API still returns an items array so the frontend can work with a simple structure.

3.3.7 Sales Page

The sales page was developed for customer-side transactions. It records sale headers, customer information, customer PO reference, sale items, documents, VAT totals, discounts, and delivery-related statuses. Sale item statuses include pending, packed, shipped, delivered, cancelled, and returned. The backend validates stock before a sale can commit stock. When the sale item is packed, shipped, or delivered, stock allocations are created from received purchase layers.

3.3.8 Billing Notes Page

The billing notes page was developed for customer receivable tracking. It groups eligible sales into billing notes and allows the user to track whether each line has been received. The backend controls eligibility so the same sale is not selected into more than one active billing note. The page also shows summary values, status filters, expected payment dates, actual payment dates, bank references, and linked credit note information.

3.3.9 Credit Notes Page

The credit notes page was developed for adjustments after a sale is cancelled or returned. The backend provides eligible sale lines, and the credit note stores customer information, sale reference, optional billing note link, line items, and total credited amount. This keeps the credit note connected to the original sale item while still preserving a readable credit document.

3.3.10 Products Page

The products page was developed as the main product catalog. It supports product creation, editing, disabling, deleting where allowed, SKUs, previous SKUs, names, category links, unit conversions, reorder levels, product pictures, supplier links, average cost information, and product transaction history. Products with transaction history can be disabled instead of removed, which protects old purchase, sale, and quotation records.

3.3.11 Categories Page

The categories page was developed to organize products in a parent-child tree. The full category tree is shown in one view because hierarchy is important for understanding the catalog. Categories are stored separately from products, and products reference categories through foreign keys while also keeping a category name snapshot for compatibility and readability.

3.3.12 Supplier Page

The supplier page was developed for maintaining supplier master data. It stores company name, locations, email addresses, telephone numbers, taxpayer ID, branches, shipping addresses, remarks, payment terms, and procurement contact details. Supplier records are reused in purchases, quotations, payment batches, and product-supplier sourcing links.

3.3.13 Customer Page

The customer page was developed for maintaining customer master data. It stores company name, locations, emails, telephone numbers, taxpayer ID, branches, shipping addresses, remarks, and billing/payment term information. Customer records are reused in sales, quotations, billing notes, and credit notes.

3.3.14 Login and Settings Support

The frontend includes login-related behavior through the authentication context and API client. The backend provides Simple JWT login, refresh, and authenticated profile endpoints. The frontend stores tokens in local storage or session storage depending on the login option, refreshes access tokens, and logs out when authentication expires. The backend-wide requirement for authentication can be turned on through the INVENTORY_REQUIRE_AUTH environment setting.

3.4 System Architecture

The system uses a three-tier web architecture. React is responsible for the user interface, Django REST Framework provides the API and business rules, and MySQL stores the relational records. The frontend does not connect directly to the database. Every important action goes through the API.

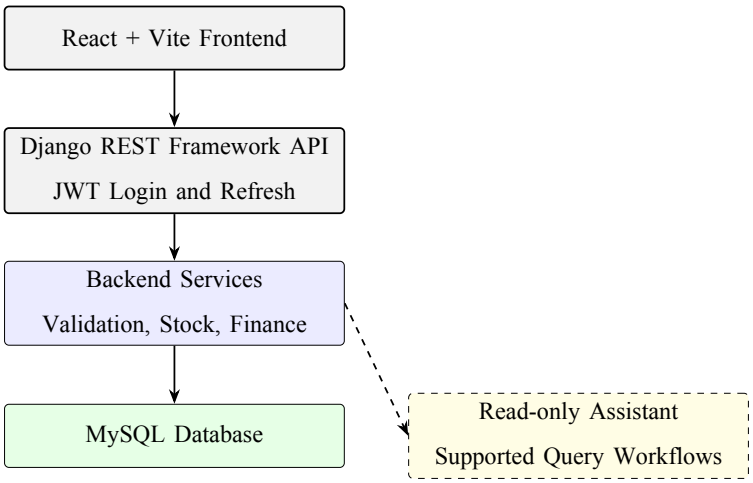

Figure 3.1 System architecture overview

Table 3.2 Architecture Components

Layer	Implementation	Main Responsibility
Frontend	React 18 and Vite	Builds the tabbed interface, forms, filters, tables, modals, and bilingual text display.
API	Django REST Framework	Receives requests, serializes data, applies filters, supports pagination, and returns JSON responses.
Business logic	Django service functions	Handles stock calculation, FIFO allocation, dashboard summaries, payable synchronization, eligibility checks, and assistant context.
Database	MySQL with Django ORM	Stores master data, transaction records, line items, documents, allocations, and finance records.
Authentication	Simple JWT	Provides login, refresh token, and authenticated profile support.

3.5 Frontend Methodology

The frontend was developed with React components, hooks, and helper files. Large pages are split into smaller sections, such as form details, line items, totals, directory tables, and detail modals. This made the code easier to manage because a purchase form, for example, has different concerns from a purchase history table.

The API client in frontend/src/api.js centralizes communication with the back-

end. It builds query strings, sends JSON or form-data requests, attaches bearer tokens, retries a request after token refresh, and dispatches an authentication-expired event when refresh fails. The data-loading hooks keep full lookup data separate from paginated directory rows, so forms can still access reference data while history pages can use pagination.

The frontend also keeps a mock-data fallback for development and demonstration. This fallback is useful when the backend is not running, but it is not treated as the final authority. The backend remains the source of truth for stock validation, eligibility, and financial synchronization.

3.6 Backend and API Methodology

The backend was developed as a Django project with one main inventory application. Django REST Framework viewsets provide CRUD endpoints for products, categories, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes. Separate API views provide dashboard summaries, dashboard segments, lookup lists, product history, product stock layers, eligibility lists, and chat responses.

Table 3.3 Main API Endpoint Groups

Endpoint Group	Purpose
/api/products/, /api/categories/	Product catalog and category tree management.
/api/suppliers/, /api/customers/	Business partner master data management.
/api/purchases/, /api/sales/	Purchase and sales transaction workflows, including line items and documents.
/api/quotations/	Quotation records with normalized line items and supplier options.
/api/billing-notes/, /api/payment-batches/, /api/credit-notes/	Customer receivable, supplier payable, and credit note workflows.
/api/eligibility/...	Backend-controlled lists for billing-note sales, payment-batch purchases, and credit-note sale lines.
/api/dashboard/, /api/dashboard/segment/	Dashboard summary and segmented operational reports.
/api/products/<id>/history/ and /stock-layers/	Product transaction history and received stock layer details.
/api/chat/	Read-only assistant answers for supported questions.
/api/auth/login/, /refresh/, /me/	Login, token refresh, and authenticated user profile.

Pagination is opt-in. If a page parameter is not sent, list endpoints can still return a plain array for compatibility with existing frontend logic. When pagination is requested, the response includes count, next, previous, page, page_size, total_pages, and results.

3.7 Database Design Methodology

The database was designed as a relational MySQL schema managed by Django migrations. Master data is separated from transaction data. For example, products, suppliers, customers, and categories are stored once and then referenced by purchases, sales, quotations, billing notes, payment batches, and credit notes.

The system also keeps snapshot fields inside transaction records. A sale item stores product name, SKU, quantity, price, and amount. A purchase stores supplier name and line item details. These snapshot values are important because old business documents should remain readable even if a product, supplier, or customer is edited later.

Table 3.4 Primary Database Model Groups

Group	Models
Master data	Category, Supplier, Customer, Product, ProductPicture, ProductUnitConversion, ProductSupplier
Purchasing	Purchase, PurchaseItem, PurchaseDocument
Sales	Sale, SaleItem, SaleItemAllocation, SaleDocument
Quotation	Quotation, QuotationItem, QuotationItemSupplier
Finance	BillingNote, BillingNoteLine, PaymentBatch, PaymentBatchLine, CreditNote, CreditNoteLine

3.8 Business Workflow Design

The main workflow starts from master data. Products belong to categories and are used in purchase, sale, and quotation line items. Purchases bring stock into the system only when purchase items are received. Sales commit stock only when sale items are packed, shipped, or delivered. This is why the system does not treat product stock as a manually edited field.

Quotations are used before purchase or sales records are created. Billing notes are created from eligible sales for customer receivable tracking. Payment batches are created from eligible purchases for supplier payable tracking. Credit notes are created from eligible cancelled or returned sale lines.

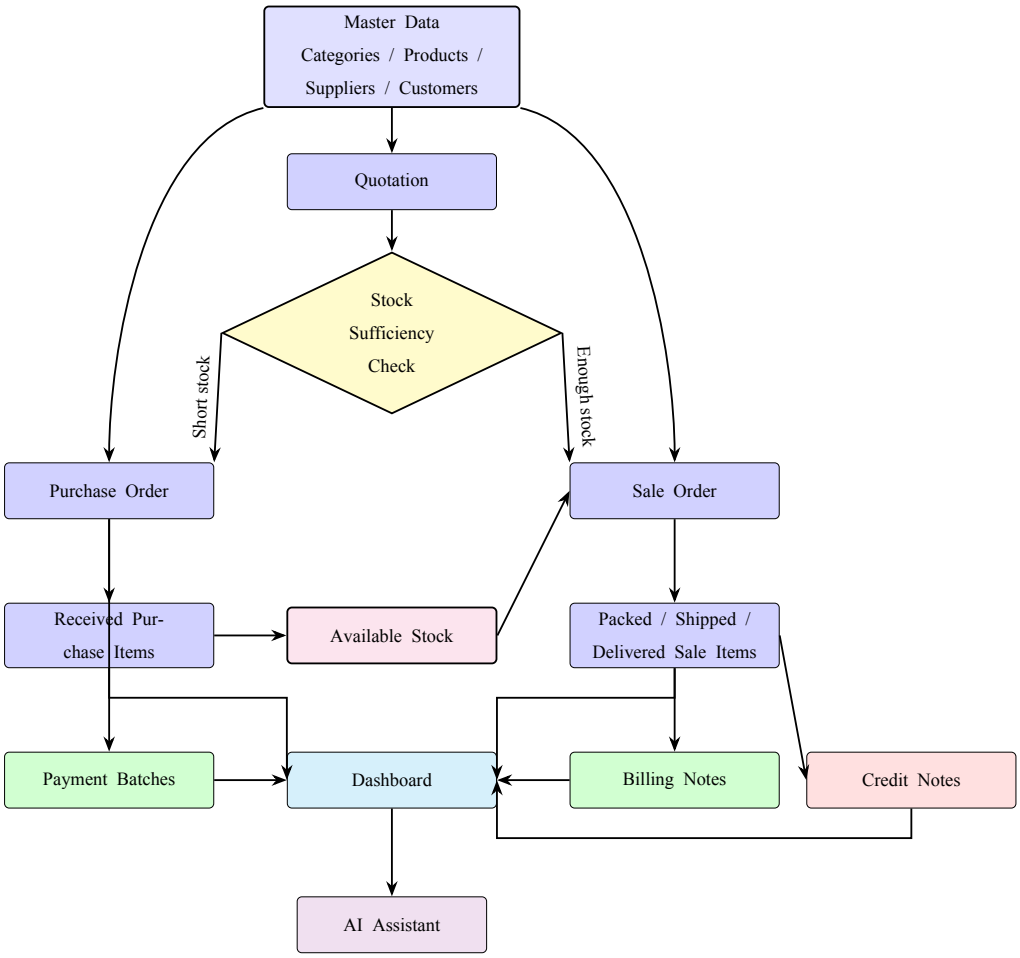

Figure 3.2 Business workflow design

3.9 Business Calculations and Rules

This section describes the main calculation logic that is implemented in the web application. Some totals are calculated in the frontend for immediate display, while the backend stores the submitted totals and applies the authoritative business rules for stock, eligibility, and payable synchronization.

3.9.1 Unit Conversion and Line Amounts

Products can have different purchase or sale units. The backend normalizes transaction quantities into the product base unit.

$$\text{Base Quantity} = \text{Quantity} \times \text{Conversion Factor}$$

Purchase and sale line amounts are calculated from quantity, unit price or unit cost, and any percentage discount chain. The frontend clamps each discount between 0% and 100%.

$$\text{Discounted Amount} = \text{Quantity} \times \text{Unit Price} \times \prod (1 - \frac{\text{Discount Percent}}{100})$$

For purchases, *Unit Price* means unit cost. For sales and quotations, it means sale price. Sales also apply the bill-level discount to the line amount when the sale total is calculated.

3.9.2 VAT Calculation

The purchase, sale, and quotation forms support VAT-included and VAT-not-included modes using a 7% VAT rate.

$$\text{VAT Rate} = 0.07$$

For VAT-included transactions:

$$\text{Total Before VAT} = \frac{\text{Item Total}}{1 + 0.07}$$

$$\text{VAT Amount} = \text{Item Total} - \text{Total Before VAT}$$

For VAT-not-included transactions:

$$\text{VAT Amount} = \text{Item Total} \times 0.07$$

$$\text{Grand Total} = \text{Item Total} + \text{VAT Amount}$$

3.9.3 Current Stock

The backend calculates product stock from transaction status instead of storing a manual stock number on the product.

$$\text{Current Stock} = \max(0, \text{Received Purchase Base Quantity} - \text{Committed Sale Base Quantity})$$

Received purchase quantity comes only from purchase items with received status. Committed sale quantity comes only from sale items with packed, shipped, or delivered status. Pending sales are shown separately as demand, but they do not reduce current stock until they reach a stock-deducting status.

3.9.4 Stock Validation and Allocation

Before a sale is allowed to commit stock, the backend compares requested committed quantity with available stock.

$$\text{Stock Issue exists if Requested Quantity} > \text{Available Quantity}$$

When stock is committed, the backend creates sale item allocations from received purchase layers. If the user does not choose stock layers manually, the backend uses first-in, first-out allocation. The received layers are ordered by received date, purchase transaction date, purchase creation date, and item ID.

$$\text{Allocation Quantity} = \min(\text{Remaining Sale Quantity}, \text{Layer Available Quantity})$$

Manual allocation is also supported, but the selected layers must match the product, must already be received, must have enough remaining quantity, and must add up exactly to the sale item base quantity.

3.9.5 Inventory Dashboard Calculations

The backend stock report calculates several useful inventory values for the dashboard and inventory page.

$$\text{Average Unit Cost} = \frac{\text{Received Purchase Value}}{\text{Received Purchase Units}}$$

$$\text{Average Daily Demand} = \frac{\text{Sales History Units}}{\text{Sales History Days}}$$

$$\text{Safety Stock} = \text{Average Daily Demand} \times \text{Safety Stock Days}$$

$$\text{Calculated Reorder Level} = \lceil \text{Lead Time Demand} + \text{Safety Stock} \rceil$$

$$\text{Recommended Restock} = \max(0, \text{ Reorder Level} + \text{Pending Sales} + \text{Oversold Units} - \text{Available Stock} -$$

$$\text{Stock Value} = \text{Available Stock} \times \text{Average Unit Cost}$$

The inventory page also uses supplier cost history to show supplier options. Supplier options are ranked by the most recent unit cost and order count, so the user can see a practical reorder source.

3.9.6 Purchase Payable and Payment Batch Calculations

The backend keeps the purchase grand total as the original document total, but it calculates a separate payable amount when purchase items are cancelled.

$$\text{Payable Total} = \text{Grand Total} \times \frac{\text{Non-cancelled Line Amounts}}{\text{All Line Amounts}}$$

This allows the system to keep the original purchase document readable while still showing the correct amount that should be paid to the supplier. Payment batch totals are calculated from their line amounts.

$$\text{Payment Batch Total} = \sum \text{Payment Batch Line Amount}$$

Unpaid payment batch lines are synchronized when the related purchase payable total changes. Paid lines are left unchanged because they represent a completed financial record.

3.9.7 Billing Note and Credit Note Calculations

Billing note totals come from selected eligible sales. Credit notes store line amounts from eligible cancelled or returned sale items. When a billing note has active credit notes, the net amount shown in the billing detail is reduced by the credit total.

$$\text{Billing Net Amount} = \text{Billing Note Total} - \text{Active Credit Note Total}$$

This makes the receivable amount clearer when a customer has a return or cancellation adjustment.

3.9.8 Cashflow and Net Position

The dashboard cashflow segment groups open receivables and open payables by due date. It also calculates the open net position.

$$\text{Open Net Position} = \text{Open Accounts Receivable} - \text{Open Accounts Payable}$$

Overdue accounts receivable are open billing notes with expected payment dates before today. Overdue accounts payable are open payment batches with planned payment dates before today.

3.9.9 Quotation Validity

Quotation validity is calculated from the quotation date and validity setting. If the validity mode is calendar days, the valid-until date is the quotation date plus the selected number of days. If the validity mode is business days, weekends are skipped. The no-valid-date option stores no valid-until date.

3.10 Validation and Security Design

Validation is handled mainly by the backend. The frontend helps the user by showing form messages and previews, but direct API calls still need to be safe. The backend validates stock availability, active product usage, allocation rules, document eligibility, quotation validity dates, and finance document selection rules.

Security support is implemented with Simple JWT. The backend exposes login, token refresh, and authenticated profile endpoints. The frontend stores and refreshes tokens, then attaches the access token to API requests. In local development, the backend can allow unauthenticated access depending on INVENTORY_REQUIRE_AUTH; when that setting is enabled, API access requires authentication.

3.11 Testing Methodology

Testing was focused on backend behavior because the backend owns the most important business rules. The test suite checks pagination, filters, product pictures, op-

erational data commands, stock validation, FIFO allocation, manual allocation, purchase item status changes, quotation normalization, reference numbers, lookup endpoints, eligibility endpoints, credit notes, purchase payable synchronization, seed data, and assistant answer alignment.

Table 3.5 Testing Summary

Test Area	**What It Checks**
Pagination and filtering	Paginated responses, page size limits, search, status filters, date filters, and product or partner filters.
Product data	Product picture upload, active/disabled behavior, lookup data, stock filter, average cost, and product history.
Stock validation	Sales cannot commit more stock than available received purchase layers.
Stock allocation	FIFO allocation, manual allocation, and stock layer remaining quantity are handled correctly.
Finance eligibility	Billing notes, payment batches, and credit notes only use eligible records.
Financial synchronization	Purchase payable totals and payment batch line amounts stay consistent when purchases change.
Assistant scope	Supported questions return data-based answers, while unsupported questions are rejected as out of scope.

3.12 Tools and Technologies Used

Table 3.6 Tools and Technologies

Category	Tool or Technology	Use in This Project
Frontend framework	React 18	Builds the web application screens and reusable UI components.
Frontend build tool	Vite	Runs the local development server and production build process.
Backend framework	Django 5	Provides project structure, ORM, migrations, settings, and backend application logic.
API framework	Django REST Framework	Provides serializers, viewsets, API views, JSON responses, parsers, and API tests.
Database	MySQL	Stores relational inventory, transaction, and finance data.
Authentication	Simple JWT	Provides access and refresh token authentication.
AI integration	OpenAI API with local fallback	Supports read-only assistant answers when configured, with local summary fallback when no API key is available.
Styling	CSS files by domain	Provides the compact inventory system interface.
Testing	Django TestCase and DRF APITestCase	Validates backend business rules and API behavior.

CHAPTER 4
RESULTS

This chapter explains the result of the implemented Inventory Management System. The result is based on the current codebase and the pages that are already implemented in the web application. The detailed development method and calculation logic were explained in Chapter 3, so this chapter focuses on what the completed system can do.

4.1 Completed System Overview

The final system is a full-stack web application for an SME trading business. It supports the flow from product setup to purchasing, selling, billing, supplier payment, credit note adjustment, inventory checking, dashboard reporting, and limited AI-assisted questions.

The main result is that the system connects business documents together instead of keeping them as separate records. A product can appear in purchases, sales, quotations, inventory reports, and product history. A sale can later become part of a billing note or a credit note. A purchase can later become part of a payment batch. This connection makes the system more useful than a simple product list.

4.2 Implemented Modules

Table 4.1 Implemented Workspace and Transaction Modules

Module	Implemented Result	Status
Dashboard	Shows stock, finance, cashflow, product movement, and order coverage summaries.	Completed
Inventory	Shows backend-calculated stock rows, reorder need, supplier options, product details, and quick purchase-order preparation.	Completed
AI Chat	Answers supported read-only questions about stock, fulfillment, partner summaries, AR/AP exceptions, and document references.	Completed
Purchases	Supports purchase records, line items, receiving status, documents, discounts, VAT totals, payable total, and payment batch links.	Completed
Payment Batches	Groups eligible purchases for supplier payment and tracks paid or unpaid lines.	Completed
Quotations	Supports quotation records, line items, supplier options, validity settings, VAT totals, and conversion to purchase or sale flow.	Completed
Sales	Supports sales records, stock validation, stock allocation, delivery status, documents, VAT totals, billing links, and credit links.	Completed

Table 4.2 Implemented Finance and Master Data Modules

Module	Implemented Result	Status
Billing Notes	Groups eligible sales for customer receivable tracking and tracks received or outstanding lines.	Completed
Credit Notes	Creates credit notes from eligible cancelled or returned sale items.	Completed
Products	Supports product CRUD, SKUs, pictures, unit conversions, reorder levels, active status, supplier links, and product history.	Completed
Categories	Supports category records with parent-child hierarchy.	Completed
Suppliers	Supports supplier profile, contact, branch, shipping address, procurement contact, and payment term data.	Completed
Customers	Supports customer profile, contact, branch, shipping address, and billing term data.	Completed

4.3 Screenshot Placeholders

The following figures are reserved for the final screenshots. They should be replaced with real screenshots from the completed application after the final demo data is approved.

TODO: Insert screenshot of the dashboard page showing stock and finance summary cards.

Figure 4.1 Dashboard page

TODO: Insert screenshot of the inventory page showing stock rows, filters, and reorder information.

Figure 4.2 Inventory page

TODO: Insert screenshot of the product form showing product details, unit conversions, and product picture support.

Figure 4.3 Product management form

TODO: Insert screenshot of the purchase workflow showing purchase items and receiving status.

Figure 4.4 Purchase workflow

TODO: Insert screenshot of the sales workflow showing delivery status and stock allocation.

Figure 4.5 Sales workflow

TODO: Insert screenshot of the quotation page showing quotation items and supplier options.

Figure 4.6 Quotation workflow

TODO: Insert screenshot of the billing note page showing eligible sales or billing note detail.

Figure 4.7 Billing note workflow

TODO: Insert screenshot of the payment batch page showing eligible purchases or payment batch detail.

Figure 4.8 Payment batch workflow

TODO: Insert screenshot of a supported read-only assistant response, such as stock status or reference lookup.

Figure 4.9 AI inventory assistant

4.4 Results by Workflow

4.4.1 Master Data Results

The master data pages were completed for products, categories, suppliers, and customers. Products can store SKUs, previous SKUs, names, categories, stock base units, default purchase and sales units, reorder levels, pictures, supplier links, and active status. Categories support hierarchy through parent categories. Suppliers and customers store contact, branch, shipping address, taxpayer, remark, and payment term information.

This result is important because the transaction pages depend on these records. Purchases need suppliers and products. Sales need customers and products. Quotations can use products, suppliers, and customers. The master data pages therefore work as the foundation of the system.

4.4.2 Purchasing and Stock Results

The purchase workflow was completed with purchase headers, purchase line items, item receiving status, expected delivery date, received date, uploaded documents, VAT fields, discounts, grand total, and payable total. A received purchase item increases available stock, while pending and cancelled purchase items do not.

The stock result is calculated by the backend. The inventory and dashboard pages show current stock, pending purchase units, delayed purchase units, pending sales units, recommended restock, stock value, and supplier options. This makes the inventory page useful for daily checking because the user does not need to manually combine purchase and sales records.

4.4.3 Sales Results

The sales workflow was completed with sale headers, customer information, customer purchase order reference, sale line items, delivery statuses, uploaded documents, VAT fields, discounts, billing note links, credit note links, and source quotation links.

The most important result in the sales module is server-side stock validation. A sale can stay as a draft even if there is not enough stock, but it cannot commit unavailable stock when the sale item becomes packed, shipped, or delivered. When stock is committed, the backend creates allocation records from received purchase layers. This gives better traceability between stock received from suppliers and stock sold to customers.

4.4.4 Quotation Results

The quotation workflow was completed with quotation date, validity settings, customer and supplier references, shipping date, payment terms, VAT mode,

total values, line items, and supplier options for each line. Quotation lines are stored in relational tables, not as one large JSON field. The frontend can also start purchase or sale creation from a quotation.

4.4.5 Finance Results

The finance workflow was completed through billing notes, payment batches, and credit notes. Billing notes group eligible sales for customer receivables. Payment batches group eligible purchases for supplier payables. Credit notes are created from eligible cancelled or returned sale items.

The backend eligibility endpoints are one of the useful results of this implementation. They reduce the chance of selecting the same sale into multiple active billing notes, selecting the same purchase into multiple active payment batches, or crediting the same returned/cancelled sale item more than once.

4.4.6 Dashboard and Inventory Results

The dashboard and inventory pages show the business status without loading every transaction into the frontend. The backend prepares stock report rows, finance summaries, cashflow buckets, product movement, popular products, and order coverage data. This result makes the interface faster and easier to use because the frontend receives summary data that is already prepared for display.

4.4.7 AI Assistant Results

The AI assistant was implemented as a limited read-only assistant. It can answer supported questions about low stock, restock scope, net position, order coverage, customer summary, supplier summary, overdue or exception issues, and line items for a known reference. The tests also show that questions outside these supported workflows are rejected as outside the current assistant scope.

4.5 Feature Evidence Summary

Table 4.3 Evidence of Implemented Features

Feature	Evidence in the Implemented System
REST API communication	The frontend API client calls endpoints for dashboard, lookups, products, purchases, sales, quotations, billing notes, payment batches, credit notes, authentication, and chat.
JWT authentication support	The backend provides login, refresh, and profile endpoints, and the frontend stores and refreshes access tokens.
Backend stock validation	The sale workflow calls backend validation before stock-deducting sale statuses are committed.
FIFO and manual allocation	The backend creates sale allocations from received purchase layers and validates manual layer selections.
Finance eligibility	Backend endpoints return only eligible sales for billing notes, eligible purchases for payment batches, and eligible sale lines for credit notes.
Normalized quotation items	Quotation lines and supplier options are stored in relational tables while the API still returns a simple items array to the frontend.
Backend dashboard summaries	The dashboard and inventory pages use backend-prepared summary data instead of calculating all operational metrics from full transaction lists in the browser.
Assistant scope control	The chat service answers only supported read-only workflows and rejects unsupported requests as outside scope.

4.6 Key Findings

The first key finding is that inventory stock should be derived from transactions, not typed manually into the product record. In this system, stock depends on received purchase items and committed sales. This makes stock more reliable because it follows the actual business documents.

The second key finding is that finance documents need backend eligibility rules. Billing notes, payment batches, and credit notes are easy to duplicate by mis-

take if the frontend is the only place checking them. Moving the rules to the backend makes the workflow safer.

The third key finding is that historical snapshot fields are useful. Even when products, suppliers, and customers are stored as master data, old transactions still need to keep names, SKUs, prices, and totals so that the document remains understandable later.

4.7 Discussion of Objective Achievement

The first objective was to record purchase and sales transactions with process status and attachment support. This objective is achieved because purchases and sales both support line items, statuses, document uploads, and transaction detail views.

The second objective was to provide inventory stock information and calculation support. This objective is achieved through backend stock calculations, inventory page stock rows, dashboard summaries, product history, stock layers, reorder information, and recommended restock values.

The third objective was to integrate an AI chatbot connected to backend data. This objective is achieved within the current implemented scope. The assistant can answer supported read-only operational questions from backend data, but it is not a general-purpose chatbot and it does not change system records.

4.8 User Survey Evaluation

At the current stage, the user survey evaluation has not been conducted because no users have evaluated the software yet. This section is kept as a placeholder for the final blackbook after real users or classmates have tested the system.

Table 4.4 User Survey Evaluation Placeholder

Evaluation Item	Result
Number of participants	TODO: Insert number of users who evaluated the system.
User group	TODO: Insert participant type, such as students, SME staff, or project reviewers.
Evaluation method	TODO: Insert survey method, rating scale, and evaluation questions.
Average score	TODO: Insert average usability or satisfaction score after the survey is completed.
User comments	TODO: Insert summarized feedback from users.

4.9 Limitations Found During Development

The system is functional, but some limitations remain:

- Final screenshots still need to be inserted into the report.
- User survey results are currently placeholders because no users have evaluated the software yet.
- Backend-wide authentication is configurable, so the final deployment setting must be checked before production use.
- The frontend still keeps mock-data fallback behavior for demonstration, so final evaluation should use backend-connected data.
- Large-scale production performance testing with many users and a very large database was not completed in the current codebase.

CHAPTER 5
CONCLUSION

5.1 Brief Project Overview

The Inventory Management System was developed as a web application for SME trading businesses that buy products from suppliers and resell them to customers. The system covers the main daily work of this type of business, including product records, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, credit notes, inventory checking, dashboard summaries, and limited AI-assisted questions.

The project was built with a React and Vite frontend, a Django REST Framework backend, and a MySQL database. The main idea of the system is to connect stock, documents, and finance records together so the user can follow the business workflow from purchase to sale and payment.

5.2 Objective Summary

The project objectives focused on three main goals: recording purchase and sales transactions with status and document attachment support, showing useful inventory stock information, and adding an AI chatbot that can answer supported questions from backend data.

The implemented system meets these goals within the current project scope. Purchases and sales support status tracking and documents. Inventory stock is calculated from received purchases and committed sales. The assistant can answer supported read-only questions, but it cannot create or edit records.

5.3 System Design Summary

The system was designed as a three-layer web application. The frontend handles the pages and user interaction. The backend handles APIs, validation, stock rules,

eligibility rules, dashboard summaries, and assistant context. The database stores the actual business records using relational tables.

This design was useful because it keeps important business rules on the backend. The frontend can make the system easier to use, but the backend still controls the rules that protect stock, billing, supplier payments, and credit notes.

5.4 Implementation Summary

The frontend implementation includes pages for dashboard, inventory, AI chat, purchases, payment batches, quotations, sales, billing notes, credit notes, products, categories, suppliers, and customers. It also includes a shared API client, authentication handling, pagination support, filters, modals, document references, bilingual text, and mock-data fallback for development.

The backend implementation includes Django models, serializers, viewsets, service functions, dashboard endpoints, lookup endpoints, eligibility endpoints, authentication endpoints, management commands, and backend tests. The most important backend logic includes stock calculation, stock allocation, sale stock validation, purchase payable calculation, finance eligibility, and assistant answer preparation.

5.5 Result Summary

The final result is a working inventory management system that connects the main trading workflows. A received purchase can increase stock, a committed sale can reduce stock, a billing note can track money expected from a customer, and a payment batch can track money that needs to be paid to a supplier.

The dashboard and inventory pages make the system easier to review because they summarize stock and finance information in one place. The assistant adds another way to ask about supported operational information, but it stays within a narrow read-only scope.

5.6 Key Findings

The main finding from this project is that stock should be calculated from transactions instead of being manually typed into a product field. Received purchases and committed sales give a clearer picture of what stock is really available.

Another finding is that backend validation is necessary. The frontend can guide the user, but the backend must still reject invalid stock commitments and invalid finance document selections.

The project also showed that historical snapshots are important. Old business documents should still show the product name, SKU, supplier name, customer name, prices, and totals that existed when the document was created.

5.7 Obstacles and Challenges

The main challenge was connecting many workflows without creating inconsistent data. A change in purchase status can affect stock and supplier payment. A change in sale status can affect stock, billing, and credit notes.

Another challenge was keeping the system understandable for daily use while still applying strict backend rules. The user interface needed to stay practical, but the backend still had to protect the data.

The last challenge was preparing enough final evidence for the report. Screenshots and final test logs still need to be added, and the user survey section must remain as a placeholder until users evaluate the software.

5.8 Future Work

Future work should stay focused on practical improvements:

1. Add final approved screenshots, test logs, and real user survey results after users evaluate the software.
2. Add role-based permissions if the system will be used by staff with different responsibilities.
3. Add clearer audit history for important status changes such as purchase receiving, sale delivery, billing receipt, and supplier payment.

4. Improve deployment testing with larger data and more users before real production use.

REFERENCES

[1] K. Lappalainen and S. Narayanan. Aisha: A custom AI library chatbot using the ChatGPT API. *Journal of Library and Information Services in Distance Learning*, pages 1--15, 2023. doi: 10.1080/19322909.2023.2221477.

[2] H. Mittal and M. Arora. A full-stack web solution for online FMCG (fast-moving consumer goods) management system using MERN: Design, implementation and evaluation. *International Journal of Latest Trends in Engineering, Management and Applied Sciences (IJLTEMAS)*, 2024.

[3] M. Muller. *Essentials of Inventory Management*. AMACOM, New York, NY, 2 edition, 2011.